

Shared Data Integrity Audit Scheme Based on CP-ABE and Deduplication for Cloud Storage

Xiaodong Yang, *Member, IEEE*, Ruiting Li[✉], Songyu Li[✉], Na Wang, and Caifen Wang

Abstract—With the rapid growth of cloud storage user scale, there are dual challenges of resource waste and security hazards in massive data storage: on the one hand, redundant data exacerbates storage overhead; on the other hand, unauthorized access exacerbates the possibility of data leakage or tampering. To this end, this paper designs a shared data auditing scheme that supports fine-grained access control and cloud deduplication. First, combining Random Convergent Encryption (RCE), Proof-of-Ownership (PoW) technology, and introducing a certificate-less cryptosystem to securely deduplicate data in the cloud while eliminating certificate management and key escrow risks. Second, the data is encrypted using Ciphertext-Policy Attribute-Based Encryption (CP-ABE) technology, and only authorized users can decrypt the data based on attribute matching, which can effectively resist the risk of data leakage due to unauthorized access. At the same time, we designate an auditor to carry out data integrity auditing to prevent malicious audit behavior. Finally, Security and performance evaluations show that the scheme we've proposed is quite efficient and competitive.

Index Terms—Data integrity auditing, certificate-less signature, data deduplication, attribute-based encryption, designated auditor.

I. INTRODUCTION

WITH the ongoing advancement of cloud computing, the cloud has emerged as the ideal platform for users to share their data. A growing number of data owners (DOs) is entrusting their data to the cloud and sharing it with users [1]. Although cloud storage offers tremendous convenience to DOs, they lose authority over their data, which can lead to data security and privacy issues [2], [3]. Current research in the cloud storage sector focuses on two aspects: first, how to efficiently identify and eliminate duplicate data to optimize

storage resource utilization [4]; and second, how to forbid unauthorized access and reduce the risk of data leakage while ensuring the convenience of data sharing [5].

On the one hand, to decrease the storage overhead, data deduplication technology has become the current research hotspot in the field of cloud storage [4]. In the cloud storage environment, a large amount of duplicate data occupies valuable storage space, which not only causes a waste of resources but also increases the complexity of indexing and maintenance, which in turn weakens the reliability of the system [6], [7]. Deduplication technology effectively reduces the actual amount of stored data by accurately identifying and removing duplicate data blocks [8], [9]. To achieve this goal, the researchers introduced a PoW approach based on zero-knowledge proofs. They used a challenge-response model to validate data holding rights, and combined with RCE encryption to achieve data deduplication [10]. However, in many existing deduplication schemes, the keys are usually centrally managed by a cloud service provider (CSP) or a third-party organization, an approach that carries the risk of data leakage [4], [6], [8], [9], [11], [12]. Traditional public key infrastructures (PKI) rely on certificate authorities to verify user identities and the validity of keys, but this process is complex and costly. How to achieve efficient deduplication while ensuring data security remains a technical challenge.

On the other hand, to realize secure access control to stored data, the Attribute-Based Encryption (ABE) technique is proposed and applied [13]. In the ABE scheme, different attributes can be associated with the identity of the data user. The DO can define the access policy before storing data and use ABE to realize the information hiding of the transmitted information [14]. Later, within the CP-ABE scheme, it is required that DO can define fine-grained access policies when encrypting data and embed these policies in the ciphertext. Only authorized users who conform to the access policies can decrypt and recover data, thus enabling precise control over data access [5], [15], [16]. This model denies access to unauthorized users, which protects the privacy of data and allows DO to set access rights based on the user's identity flexibly attributes [14]. However, it has a clear limitation: the failure to integrate data deduplication technology restricts its application in optimizing storage efficiency. To address this limitation, our scheme ingeniously combines two mechanisms, effectively tackling the challenge of balancing security and efficiency.

Meanwhile, to ensure data integrity while reducing the computational burden on DOs, a third-party auditor (TPA) is introduced to enable data integrity auditing [17], [18].

Received 7 May 2025; revised 29 October 2025 and 15 December 2025; accepted 29 December 2025. Date of publication 1 January 2026; date of current version 13 January 2026. This work was supported in part by the National Cryptologic Science Fund of China (no. 2025NCSF02040), the National Natural Science Foundation of China (no. 62362059), the Industrial Support Plan Project of Gansu Provincial Education Department (no. 2023CYZC-09), and the Key Research and Development Program of Gansu Province (no. 23YFGA0081). The associate editor coordinating the review of this article and approving it for publication was M. Li. (Corresponding authors: Ruiting Li; Na Wang.)

Xiaodong Yang, Ruiting Li, and Songyu Li are with the College of Computer Science and Engineering, Northwest Normal University, Lanzhou 730070, China (e-mail: yangxd200888@163.com; lrt13893206276@163.com; lsy1456123372@163.com).

Na Wang is with the Innovation Development and Evaluation Division, Gansu Computing Center, Lanzhou 730070, China (e-mail: shally11219@163.com).

Caifen Wang is with the College of Big Data and Internet, Shenzhen Technology University, Shenzhen 518118, China (e-mail: wangcfen126@163.com).

Digital Object Identifier 10.1109/TNSM.2025.3650222

However, such schemes generally suffer from a critical flaw: the auditing entity is predetermined and immutable. Consequently, when users intend to entrust data to a specific reputable auditor, the existing auditing process cannot be executed because of without a flexible assignment mechanism.

We designed a shared data integrity auditing scheme supporting CP-ABE encryption and deduplication based on the above questions. The scheme combines several cutting-edge technologies, such as certificate-less cryptosystems, and aims to solve the key problems in the course of cloud data storage, sharing, and auditing. First, the scheme is founded on certificate-less public key cryptosystem and utilizes aggregate key encryption to achieve data deduplication in cloud servers and save storage costs. Meanwhile, to ensure data sharing security, the scheme uses encryption technology based on CP-ABE to encrypt data. In addition, to ensure data integrity, we introduce a designated auditor (DA) mechanism. The DA is tasked with conducting periodic integrity checks on data stored on the cloud server to ensure it hasn't been tampered with or deleted. The core contributions of our scheme are as follows:

- 1) Certificate-less signature algorithms for efficient data integrity auditing: our scheme solves the integrity auditing of data stored using a certificate-less signature algorithm, avoiding the expensive cost problem in certificate management and the key escrow issues.
- 2) RCE combined with PoW for data Deduplication Technology: Combines RCE (same plaintext generates same ciphertext) and PoW for efficient deduplication. Quickly identifies redundant data through ciphertext comparison, reducing storage costs for CSP.
- 3) CP-ABE-Based Fine-Grained Access Control: Encryption policies are defined through access trees, and CP-ABE ciphertext is constructed accordingly. Only authorized users whose attributes satisfy the preset policy are granted decryption capability, thereby effectively preventing data leakage risks caused by unauthorized access.
- 4) We proposed a scheme that would require DAs to carry out public audits, and only DAs with the correct identity could participate in audit assignments.
- 5) We evaluate the communication and computational overhead of our scheme against several auditing schemes. The performance analysis demonstrates that our scheme offers greater advantages and broader applicability.

II. RELATED WORK

In 2007, Ateniese et al. [19] first introduced a provable data possession (PDP) scheme, which is extensively applied in cloud storage services to safeguard data integrity. However, initial cloud auditing schemes relied on PKI, which was complex and time-consuming to do. Subsequent researchers have proposed various improvements: Yang and Yu [11] designed identity-based protocol for verifying the integrity of multi-copy data using a multi-branch authentication tree. Sahu and Nene [20] developed an identity-based cloud auditing scheme incorporating a lattice signature algorithm. Rabaninejad et al. [21] devised auditing scheme that supports

both online and offline operations. However, these schemes have key escrow problems, and to solve this problem, scholars design a certificate-less authentication auditing scheme. Huang et al. [22] reduced the computation overhead through a public auditing mechanism to realize multi-file batch auditing. Guan et al. [23] designed a secure data sharing scheme founded on the Sanitizer, which supports the attribute access policy to ensure fine-grained access authorization. However, these schemes do not mention how to counteract dishonest TPAs. With the TPA proxy integrity auditing system, Yu et al. [24] proposed the concept of zero-knowledge private data and designed an RSA-based encryption scheme, which ensures that TPAs do not have access to information about the audited data. Ren et al. [25] proposed a secure storage structure within the cloud consisting of DA, where user data are able only to be partitioned in data chunks, and the division of data chunks into data segments is not supported. Li et al. [26] devised an efficient certificate-less provable data holding mechanism in the cloud. The scheme utilizes dynamic hash tables to synchronize cloud data, and this mechanism can ensure data privacy. However, numerous duplicated resources in the present day cause privacy-preserving deduplication to face the problem of resource wastage.

In order to realize privacy-preserving deduplication, Douceur et al. [27] designed the concept of Convergent Encryption (CE), which implements encryption using the hash value of a file. Different users has the ability to encrypt an identical file to the same ciphertext. Later, CE was formally named as MLE [10]. However, the MLE algorithm is susceptible to brute force attacks. To cope with the vulnerability of MLE algorithm to brute-force attacks, some schemes introduce key servers to assist in the process of forming MLE keys to enhance security. Zhang et al. [28] implemented deduplication in a federated cloud system by using ring signing to balance security and efficiency. Zheng et al. [29] implemented deduplication of media data with the help of server-assisted MLE to expand the application scenarios of MLE. Yuan and Yu [11] took the lead in proposing a scheme that integrates deduplication and data integrity auditing, but the scheme involves a significant computational overhead, which limits its practical application to a certain extent. Xu et al. [30] utilized blockchain technology to supervise incompletely trustworthy TPA, which innovatively combines blockchain's non-tamperable characteristics with cloud storage auditing. However, in plaintext, their scheme implements deduplication and auditing, which is a security risk. Bai et al. [31] utilized Identity-Based Broadcast Encryption to reduce user-side storage overhead and optimize efficacy of storage. Subsequently, Liu et al. [32] constructed the scheme that doesn't rely on trusted proxy servers, which achieves tagged deduplication with the support of MLE keys and reduces the dependence on trusted third parties. However, when the data has a low entropy value, this scheme cannot ensure the reliability of the audit results, and Gao et al. [33] designed an enhanced scheme that replaces the MLE key in conjunction with a random key picked by the initial uploader. However, how to certify the reliability of the audit results of the subsequent uploaders is still a problem, and further research and exploration are

urgently needed. Miao et al. [8] used the ownership proof to realize the audit problem of data uploading by the successors, but ignore the problem of data sharing by unauthorized users.

To address the problem of data sharing for unauthorized users, a file sharing based on the SSE scheme proposed in [34], which stores and encrypts data that can be directly searched without prior decryption, fails to complete fine-grained access control. The scheme in [35] used the CP-ABE technique to encrypt data and policies, but the complexity of CP-ABE operations leads to overall inefficiency. Yu et al. [36] proposed an innovative approach aimed at solving the complicated key management challenges faced during cloud data integrity checking. This paper presents an attribute-based cloud data auditing mechanism, which is tasked with verifying the soundness of externally stored data via allocating a team of auditors. However, it relies on the independence and impartiality of the auditors, which may affect the objectivity and reliability of the whole auditing process if there are problems with the auditors themselves. Chen et al. [37] proposed the scheme of outsourcing the attribute revocation to the cloud servers, where the user is able to verify whether downloaded data has not been revoked or not, thus simplifying the operation. However, this scheme does not consider the user burden of ABE encryption and the problem of original data integrity verification. Goswami et al. [38] introduced the data auditing scheme founded on short signatures, which can guarantee the security and confidentiality of data integrity auditing, but the computational overhead is still large. Yao et al. [5] proposed a key-based access control strategy founded on attribute encryption, which encrypts the access control in the original data instead of the encryption of the access control by ciphertext in the original data instead of the ciphertext, which significantly enhances the computational efficiency.

In summary, most existing data integrity auditing schemes face dual challenges in shared data environments: data leakage risks and resource inefficiency. These limitations restrict their widespread deployment in practical scenarios. Consequently, there is an urgent need for an integrity auditing mechanism that simultaneously supports fine-grained access control and cloud-based data deduplication, balancing security with storage efficiency. Our proposed scheme integrates certificate-less cryptography with CP-ABE. It achieves efficient data deduplication while introducing a DA mechanism, effectively safeguarding both privacy and integrity during data sharing. Compared to existing approaches that focus solely on a single security dimension or neglect deduplication requirements, our scheme offers a more systematic and comprehensive rescheme across multiple layers, including security, privacy protection, and resource management.

Organization: Our scheme is organized as shown below: the first chapter provides a review of the current state of research and describes our main contributions. The second chapter is devoted to summarizing the related work. Preliminary knowledge appears in Section III. Section IV discusses the system model and security model of our scheme in detail. Section V puts forward the designed scheme. Section VI undertakes the security analysis. Section VII undertakes the

performance analysis of the scheme. In conclusion, the end section sums up the entire paper.

III. PRELIMINARIES

A. Bilinear Pairings

Suppose that G and G_T are two multiplicative cyclic groups of prime order q . g is the distinct initial element of the group G . $e: G \times G \rightarrow G_T$ is a bilinear pairing map with the subsequent characteristics.

- 1) Bilinearity: For $\forall u, v \in G, \forall a, b \in \mathbb{Z}_p$, it has $e(u^a, v^b) = e(u, v)^{ab}$.
- 2) Computability: For $\forall u, v \in G$, $e(u, v)$ is able to be calculated effectively.
- 3) Non-degeneracy: $\exists u, v \in G, e(u, v) \neq 1$.

B. Hardness Problem

- Computational Diffie-Hellman (CDH) assumption: Given the tuple $(g, g^a, g^b) \in G$, the superiority of algorithm A as a stochasticized algorithm with polynomial complexity for computing $g^{ab} \in G$ is negligible.
- Discrete Logarithm (DL) assumption: With the tuple $(g, g^a) \in G$ provided, the likelihood of computing $a \in \mathbb{Z}_q$ for a stochasticized algorithm with polynomial complexity A is negligible.
- Decisional Bilinear Diffie-Hellman (DBDH) problem: Under the condition that G and G_T are both multiplicative cyclic groups, g serves as the generator of G , three random numbers $r_1, r_2, r_3 \in \mathbb{Z}_q$ are chosen. Given $g, g^{r_1}, g^{r_2}, g^{r_3} \in G$ and $\mathcal{T} \in G_T$, it is computationally infeasible to decide if

$$\mathcal{T} = e(g, g)^{r_1 r_2 r_3}$$

C. Access Structure

The access tree implements conditional access protection for data keys through a tree structure. Its core components are shown in Fig. 1: The root node stores the final key, which can only be obtained by visitors meeting its minimum threshold. Non-leaf nodes serve as threshold nodes, requiring specific thresholds to be met for decrypting classified secrets. Leaf nodes bind attributes and secret values distributed by parent nodes. By setting different thresholds for threshold nodes (e.g., an “AND” gate requires all child nodes, while an “OR” gate requires only one child node).

The access tree γ is a tree structure where non-leaf nodes represent threshold gates. The logical semantics of node x are determined by its number of children cx and threshold kx : $kx = 1$ denotes an “OR” gate, while $kx = cx$ denotes an “AND” gate. Additionally, the system defines key functions for the tree structure: $pa(x)$ returns the parent node, $att(x)$ determines if it is an attribute leaf node, and $in(x)$ provides a unique sequential index for each node.

The construction of an access tree fundamentally involves distributing the root key securely through polynomial secret sharing, layer by layer along the tree structure down to leaf nodes. Decryption of the access tree is a bottom-up process: starting from a matching leaf node, the user recursively

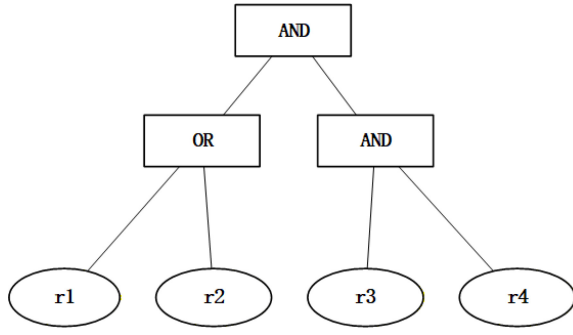


Fig. 1. Access structure.

reconstructs parent node secrets until conditionally recovering the root key.

D. CP-ABE

CP-ABE, as a cryptographic scheme for fine-grained access control, has the following core algorithmic composition:

- *Setup*(1^λ): Generate system public parameters *para* and master key *MSK*.
- *Key Generation*(*MSK*, *T*): Generate the corresponding decryption private key K_{dus} according to the cluster of user attributes *T* and the master key *MSK*, and realize the binding of attributes and keys.
- *Encrypt*(*para*, *F*, γ): The encryption algorithm takes a public parameter *PK*, the data *F* to be encrypted, and an access structure γ as inputs, and produces the ciphertext *FC*.
- *Decrypt*(*para*, *FC*, *SK*): The decryption algorithm requires the public parameter *para* and a ciphertext *FC* as inputs, with *FC* incorporating the access structure γ . Decryption is feasible and yields *F* if the user's attribute set *T* fulfills the access structure γ .

IV. SYSTEM AND SECURITY MODEL

In this section, we will introduce the system model, definition, and security model. Table I generalizes the key definitions and notations involved in the scheme.

A. System Model

Our model, presented in Fig. 2, comprises five entities: a DO, a DA, a CSP, a Key Generation Center (KGC), and a Data User (DU). The responsibilities and obligations they are required to undertake are as below.

- 1) *DO*: The DO often uploads data to the CSP for storage. In our scheme, DO includes the initial uploaders DO_{iu} and subsequent uploaders DO_{su} .
- 2) *DA*: We designed the auditors to conduct public audits, and only DAs with the correct identity were allowed to participate in audit assignments. DA operated in a semi-honest manner, diligently carrying out audit assignments but was inquisitive about the content of the data.
- 3) *CSP*: DO stores data in CSP and implements deduplication in CSP. DO delegates to DA to query CSP, CSP collects proof and replies to DA. CSP is not believed and may compromise the integrity of user data.

TABLE I
NOTATIONS AND DEFINITIONS

Notations	Definitions
T_E	Operations of exponentiation on group G
T_M	Operations of multiplication on group G
T_P	Operations of pairing on group G
S_{DU}	The user's own attributes
N_C	The number of attributes included in the ciphertext.
N_S	The minimum attributes meeting the access structure
c	Count of challenge blocks
n	Count of data blocks
$ G $	Size of the element in $ G $
$ Z_q $	Size of the element in Z_q
K_{dos}	The partial key of DO
K_{das}	The partial key of DA
K_{dus}	The partial key of DU
<i>MSK</i>	The master key
<i>MPK</i>	The system public key
e	The bilinear pairing
g	The generator of G
G, G_T	The multiplicative cyclic group
<i>FC</i>	The final encrypted ciphertext
γ	The access policy
id_{DO}	The DO's identity
id_{DU}	The DU's identity
r	The attribute value
kc_i	The convergent key
$pa(x)$	The parent node of node x in the tree.
$in(x)$	The index of node x among its sibling nodes
$att(x)$	The attribute corresponding to the leaf node
x	The root node
LS	The set of leaf nodes
k_l	The file encryption key
f_i	The file blocks of F
σ_i	The signature
SK_F	The private key for file F

- 4) *DU*: Some users need to use the data stored by DO. First need to initiate a request to CSP, CSP verifies their identity and sends the ciphertext *FC* encrypted by attributes to DU. If DU can decrypt the ciphertext to complete the data acquisition.
- 5) *KGC*: The KGC is tasked with creating partial keys for users within the integrity assurance system, using their specific identity information as the basis.

B. Definition of Our Scheme

The scheme proposes a data integrity auditing scheme supporting fine-grained access control and cloud-based deduplication. Its core process is summarized as below:

- 1) *Setup* $\rightarrow \{para\}$. The algorithm is executed by KGC and outputs the public parameter *para*.
- 2) *KeyGen* $\rightarrow \{K_{dos}, K_{das}, K_{dus}\}$. The *KeyGen* algorithm enforced by KGC to create the private key K_{dos}

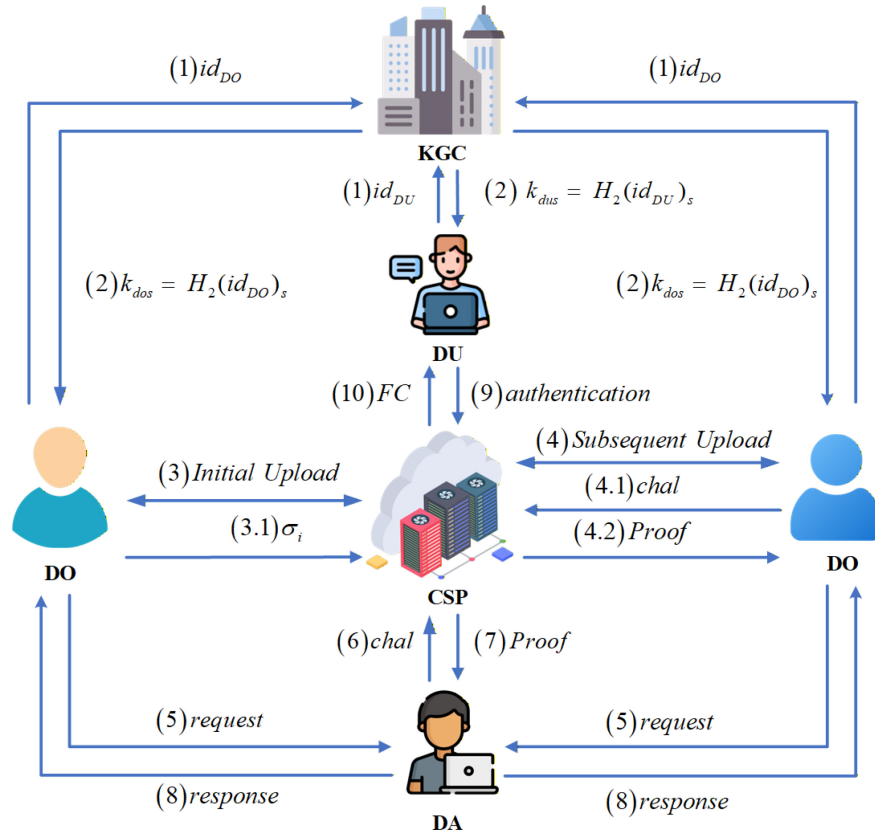


Fig. 2. System model.

of DO, the private key K_{das} of DA, and the private key K_{dus} of DU.

- 3) $CiphertextGen \rightarrow \{FC\}$. The DO executes the algorithm to generate the ciphertext FC .
- 4) $InitialUpload \rightarrow \{\sigma_i\}$. The DO runs the $TagGen$ algorithm and $Store$ algorithm to generate tags σ_i and uploads them to the CSP for storage.
- 5) $SubsequentUploads \rightarrow \{\sigma\}$. Subsequent upload steps include Proof of ownership generation ($PoW.Gen$) and Proof of ownership verification ($PoW.Ver$) executed by DO and CSP.
- 6) $ChalGen \rightarrow \{chal\}$. TPA executes the $ChalGen$ algorithm to generate stochasticized challenge $chal = (c, k_1, k_2)$, and transmits it to the CSP.
- 7) $ProofGen \rightarrow \{Proof\}$. With the index of the challenged blocks I, C as input, the algorithm outputs the corresponding proof $Proof$.
- 8) $DesignAuditor \rightarrow \{Proof\}$. The CSP designates a DA to conduct the audit and sends $Proof$ to the DA.
- 9) $ProofVerify \rightarrow \{T/F\}$. The DA executes the $ProofVerify$ algorithm to inspect the integrity of the data by verifying proofs.
- 10) $Dec \rightarrow \{F\}$. The DO implements the algorithm, which takes the ciphertext FC as input and produces F as output.
- 11) $Authentication \rightarrow \{T/F\}$. The CSP executes CP-ABE encryption to authenticate the legitimacy of the DU's identity.

C. Security Model

In the security model section, we theoretically demonstrate the correctness of the scheme, the unforgeability of the tag and proof, Indistinguishability under Choice of Plaintext Attack (IND-CPA), and anti-collusion attack firstly. Additionally, the scheme ensures data privacy protection, supports secure tag deduplication, and provides reliable data ownership proofs. This section will focus on the following five types of attackers:

- 1) Adversary $\mathcal{A}_1$: Dishonest users can't get access to the master key and partial keys, but they can replace the public key and secret values of legitimate users.
- 2) Adversary $\mathcal{A}_2$: Untrustworthy KGC that has access to master keys and partial keys, but cannot query or replace legitimate users' public keys.
- 3) Adversary $\mathcal{A}_3$: A malicious CSP that attempts to falsify proof to deceive a DA and pass integrity verification.
- 4) Adversary $\mathcal{A}_4$: The attacker is able to choose any plaintext and obtain its encrypted ciphertext, but has no direct access to the encryption key. Attempts to deduce the key or break the confidentiality of the encryption scheme.
- 5) Adversary $\mathcal{A}_5$: In cases where the CSP has tampered with or deleted portions of the data, colluding with the DA to generate a valid audit proof in order to deceive the DO.

V. PROPOSED SCHEME

This section details a shared data integrity auditing scheme supporting fine-grained access control and cloud data deduplication. Designed to address increased data leakage risks from

redundant data and unauthorized access, the scheme covers critical processes including initialization, data upload, integrity auditing, and identity authentication, while further elucidating its construction methodology and algorithmic definitions.

- 1) *Setup* $\rightarrow \{para\}$. When the safety parameter λ is given, KGC first picks two multiplicative cyclic groups G and G_T with prime order q , where q is a large prime. Subsequently, a computable bilinear pairing is selected as $e : G \times G \rightarrow G_T$, and g is the generator of G . After that, KGC defines hash functions $H_1 : \{0, 1\}^* \rightarrow G$, $H_2 : \{0, 1\}^* \rightarrow G$, $H_3 : \{0, 1\}^* \rightarrow Z_q$, $H_4 : \{0, 1\}^* \rightarrow G$, and pseudo-random functions $\pi : \{0, 1\}^* \times G \rightarrow \{0, 1\}^*$, $prf : Z_q \times Z_q \rightarrow Z_q$, $prp : Z_q \times (1, \dots, n) \rightarrow (1, \dots, n)$. The KGC picks the master key $MSK = s$, where $MSK \in Z_q$. The KGC calculates a system public key $MPK = g^s$, where $MPK \in G$. KGC defines the Lagrange coefficient $L_{k,T}(x) = \prod_{m \in T, m \neq k} (x - m)/(k - m)$, where $T \in Z_q$. KGC randomly selects three values $\alpha, \beta \in Z_q$ and computes $g^\beta, e(g, g)^\alpha$. At last, The KGC makes the system public parameters $para = \{G, G_T, q, e, g, g^\beta, e(g, g)^\alpha, H_1, H_2, H_3, H_4, \pi, prf, prp, MPK\}$ known, and keeps the master key MSK privately.

- 2) *KeyGen* $\rightarrow \{K_{dos}, K_{das}, K_{dus}\}$. The DO transmits its identity $id_{DO} \in \{0, 1\}^*$ to the KGC to acquire its partial key. After the KGC receives the identity of the DO, the KGC calculates $K_{dos} = H_1(id_{DO})^s$ and returns K_{dos} to the DO over a safe path. The DO picks the secret value $\alpha_{DO} \in Z_q$ and computes the public key $K_{dp} = g^{\alpha_{DO}}$. The DO obtains the complete key is (K_{dos}, α_{DO}) , and the same method is used to generate the complete key of the DA as (K_{das}, α_{DA}) , the DA sends $K_{tp} = g^{\alpha_{DA}}$ as well as the identity id_{DA} to the CSP. The KGC randomly selects the attribute $r \in Z_q$ from the attribute set T and computes the following algorithm. Here is the identity information id_{DU} of DU. KGC produces the key K_{dus} and transmits it to the DU.

$$K_{dus} = \begin{cases} D_1 = (g^\alpha \times H_1(id_{DU}))^{1/\beta} \\ D_m = H_1(id_{DU}) \times H_1(m)^{r_m} \\ D'_m = g^{r_m} \end{cases}$$

- 3) *CiphertextGen* $\rightarrow \{FC\}$. The DO divides the file F into n blocks $F = (f_1, f_2, \dots, f_n)$, computes the hash of the blocks $h_i = H_2(f_i)$, and selects the random key seed $ks \in Z_q$ and computes the convergent key $kc_i = \pi(ks, h_i)$. DO choose a symmetric encryption algorithm $ENC_1(\cdot)$ to encrypt each data block $C_i = ENC_1(kc_i, f_i)$, where $i \in n$. DO chooses a public key encryption algorithm $ENC_2(\cdot)$ to encrypt the convergent key $KC_i = ENC_2(K_{dp}, kc_i)$. DO randomly selects $T_1 \in Z_q$ and calculates $e(g, g)^{\alpha T_1}$, nodes $R(\gamma)$, DO randomly select a polynomial $q_R(0) = T_1$, and define the polynomial d_R in terms of points, denote the summary point of the access tree by x , where $x \in \gamma$. DO select the polynomial q_x , from the root node from the top down and set $d_x = S_x - 1$, where S_x is the secrecy threshold, for each node x , DO randomly select the polynomial d_x to perform the computation

$q_x(0) = q_{pa(x)}(in(x))$, the final encrypted ciphertext FC is generated and LS is the set of leaf nodes. Next, the DO splits the ciphertext c_i into $c_i = c_{ij} (1 \leq i \leq n, 1 \leq j \leq s)$.

$$FC = \left\{ \begin{array}{l} c_i = C_i \times e(g, g)^{\alpha T_1}, \\ \gamma, c_1 = g^{\beta T_1}, \forall y_o \in LS: c_{y_o} = g^{q_{y_o}(0)} \\ c_{y_o} = H_1(att(y))^{q_{y_o}(0)} \end{array} \right\}$$

- 4) *InitialUpload* $\rightarrow \{\sigma_i\}$. The initial user DO_{iu} retrieves the convergence key $kc_i = \pi(ks, h_i)$ and computes the $T(F) = H_3(ENC_1(Kc_i, F))$ and sends an upload request to CSP. Upon receiving an upload request, CSP examines if $T(F)$ exists. If it doesn't, it's the initial upload.

- *TagGen*: The DO_{iu} randomly picks the file encryption key $k_l \in Z_q$. The DO_{iu} chooses s values $\{a_k \in Z_q\}_{1 \leq k \leq s}$ stochastically and sets $A_k = g^{a_k}$. The DO_{iu} initially creates the private key for file F as $SK_F = H_3(k_l)$ (this encryption key will be utilized to generate the authenticator). Suppose the data identity be $\eta = H_3(k_l || kc_i)$. Subsequently, the user produces the tag for f_i by calculating:

$$\sigma_i = K_{dos} \cdot \left(H_4(\eta || SK_F || i) \cdot g^{\sum_{j=1}^s a_j c_{ij}} \right)^{\alpha_{DO}}$$

- *Store*: The DO_{iu} uploads the data F , the signature σ_i to CSP. After CSP receives it and verifies that it is correct. If the validation fails, CSP denies storage. If the data is legitimate, the CSP verifies the signature σ_i of data F .

$$e(\sigma_i, g) = e(H_1(id_{DO}), MPK) \cdot e \left(H_4(\eta || SK_F || i) \cdot \prod_{j=1}^s A_j^{c_{ij}}, K_{dp} \right)$$

- 5) *SubsequentUploads* $\rightarrow \{\sigma\}$. The CSP examines whether $T(F)$ exists. If exists, it is a subsequent upload. For the subsequent uploader DO_{su} , the CSP randomly selects $k_1, k_2 \in Z_q, c \in [1, n]$ as the challenge message and transmits the ownership challenge to DO_{su} , and upon being given the challenge message, the DO_{su} produces a proof.

- DO_{su} retrieve the convergence key $kc_i = \pi(ks, h_i)$. DO_{su} encrypts the file $C_i = ENC_1(kc_i, f_i)$ and to encrypt the convergent key $KC_i = ENC_2(K_{dp}, kc_i)$. DO_{su} generates the final encrypted ciphertext FC splits the ciphertext c_i into n blocks.

- DO_{su} generates challenge content based on challenge information $I = \{f_i\}_{i=1}^c, f_i = prp(k_1, i), C = \{\nu_i\}_{i=1}^c, \nu_i = prf(k_2, i)$. DO_{su} executes *PoW.Gen* information $u_j = \sum_{i=1}^c \nu_i c_{f_i j}, 1 \leq j \leq s, \sigma = \prod_{i=1}^c \sigma_{f_i}^{\nu_i}$.

CSP executes *PoW.Ver* algorithm and verifies $e(\sigma, g) = e(H_1(id_{DO}), \sum_{i=1}^c \nu_i, MPK) \cdot e(\prod_{i=1}^c (H_4(\eta || SK_F || i)^{\nu_i} \cdot \prod_{j=1}^s A_j^{u_j}), K_{dp})$.

- 6) *ChalGen* $\rightarrow \{chal\}$. The DA transmits $chal = (c, k_1, k_2)$ to CSP, where $c(1 \leq c \leq n)$ indicates the

challenged seed and $k_1, k_2 \in Z_q$ denote stochastic seeds.

- 7) *ProofGen* $\rightarrow \{Proof\}$. After receiving *chal* from DA, the CSP obtains the relevant proof *Proof*. In concrete terms, the CSP firstly generates the index set $I = \{f_i\}_{i=1}^c, f_i = prp(k_1, i), C = \{\nu_i\}_{i=1}^c, \nu_i = prf(k_2, i)$. CSP generated proof information.

$$\begin{cases} \sigma = \prod_{i=1}^c \sigma_{f_i}^{\nu_i} \\ \mu_j = \sum_{i=1}^c \nu_i \cdot c_{f_i j} \quad 1 \leq j \leq s. \end{cases}$$

- 8) *DesignAuditor* $\rightarrow \{Proof\}$. CSP designates the auditor information. It enters I, C, id_{DA}, k_{tp} of the DA and the public key k_{cp} , private key pair (k_{dcs}, α_{CSP}) of itself. Then CSP calculates $H_{DA} = H_1(\alpha_{CSP}, K_{tp}), I' = e(H_{DA} K_{tp}), C' = H_{DA} C$. CSP sends I', C' as the proof to DA. CSP sends the proof to the DA.
- 9) *ProofVerify* $\rightarrow \{T/F\}$. When receiving the integrity proof *Proof*, DA verifies the validity of *Proof* as shown below:

$$e(\sigma, g) = e\left(H_1(id_{DO})^{\sum_{i=1}^c \nu_i}, MPK\right) \cdot e\left(\prod_{i=1}^c H_4(\eta || SK_F || i)^{\nu_i} \cdot \prod_{j=1}^s A_j^{\mu_j}, K_{dp}\right)$$

Upon successful verification, the CSP sends T to the DO; otherwise, it sends F .

- 10) *Dec* $\rightarrow \{F\}$: When the DU intends to use data, it needs to satisfy the access policy γ and decrypt it. x is the access policy γ for generating access to the tree node set to $k = att(x)$ and if for $k \in T$, then the following algorithm is computed:

$$\begin{aligned} Dec &= (FC, K_{dus}, x) \\ &= e(D_k, C_x) / e(C'_x, D'_k) \\ &= e(H_1(id_{DU}), g)^{q_x(0)} \cdot e(H_1(k), g)^{q_x(0)r_k} / \\ &\quad e(H_1(k), g)^{q_x(0)r_k} \\ &= e(H_1(id_{DU}), g)^{q_x(0)} \\ &= F_x \end{aligned}$$

If it is $k \in T$, then computes $Dec = (FC, K_{dus}, x) = 0$. If x is a nonleaf node, then perform $Dec = (FC, K_{dus}, x)$. For a child node x_0 in x compute $Dec = (FC, K_{dus}, x_0)$ output F_{x_0} . T_{x_0} denotes an arbitrary set of child nodes, $F_{x_0} = 0$. If not exist, return 0; otherwise, compute:

$$\begin{aligned} F_x &= \prod_{x_0 \in T_x} F_{x_0}^{L_k, T'_x(0)} \\ &= \prod_{x_0 \in T_x} \left(e(H_1(id_{DU}), g)^{q_{x_0}(0)} \right)^{L_k, T'_x(0)} \\ &= \prod_{x_0 \in T_x} \left(e(H_1(id_{DU}), g)^{q_{pa(x_0)}(in(x_0))} \right)^{L_k, T'_x(0)} \\ &= e(H_1(id_{DU}), g)^{q_x(0)} \\ &= F_x \end{aligned}$$

This includes $k = in(x_0), T_x = \{in(x_0); x_0 \in T_x\}$. is calculated if the attribute satisfies the access tree for

that access policy $Dec = (FC, K_{dus}, x)$. The following results are obtained: $F_R = Dec(FC, K_{dus}, R) = e(H_1(id_{DU}), g)^{T_1}$, where R is the root. DU then continues to calculate:

$$\begin{aligned} B &= e(C_1, D_1) \\ &= e\left(g^{\beta T_1}, (g^\alpha \cdot H_1(id_{DU}))^{1/\beta}\right) \\ &= e(g, g)^{T_1 \alpha} \cdot e(g, H_1(id_{DU}))^{T_1} \end{aligned}$$

Eventually, the DO will obtain the encrypted file by decrypting the computing:

$$\begin{aligned} c_i \cdot F_R / B &= C_i \cdot e(g, g)^{\alpha T_1} \cdot e(H_1(id_{DU}), g)^{T_1} / \\ &\quad e(g, g)^{T_1 \alpha} \cdot e(g, H_1(id_{DU}))^{T_1} \\ &= C_i \quad (i \in n) \end{aligned}$$

- 11) *Authentication* $\rightarrow \{T/F\}$. The CSP produces a collection of random values: $R = \{r_1, r_2, r_3, \dots, r_n\} \in Z_q$. As the DO's access policy γ is publicly accessible, the CSP uses it to encrypt the random numbers, resulting in $E = \{E_1, E_2, \dots, E_n\} \in Z_q$, which are then sent to the DU for authentication purposes. Upon receiving E , the DU decrypts the ciphertext FC to retrieve the random value R_0 and forwards R_0 to the CSP. The CSP checks if R_0 matches R . If the verification is successful, the CSP transmits the ciphertext FC to the DU for subsequent decryption.

VI. SECURITY ANALYSIS

This section will conduct a provably secure formal analysis of our scheme based on the subsequent theorem.

Theorem 1: If the DO, CSP, KGC, and DA faithfully follow the prescribed procedures, the proof will successfully pass the DA's verification.

$$\begin{aligned} e(\sigma, g) &= e\left(\prod_{i=1}^c \sigma_{f_i}^{\nu_i}, g\right) \\ &= e\left(\prod_{i=1}^c (K_{dos} \cdot (H_4(\eta || SK_F || i) \cdot g^{\sum_{j=1}^s a_j c_{f_i j}})^{\alpha_{DO}})^{\nu_i}, g\right) \\ &= e\left(\prod_{i=1}^c K_{dos}^{\nu_i}, g\right) \cdot e\left(\prod_{i=1}^c H_4(\eta || SK_F || i)^{\nu_i} \cdot \prod_{j=1}^s H_4(\eta || SK_F || i)^{\sum_{i=1}^c \nu_i c_{f_i j}}, K_{dp}\right) \\ &= e\left(H_1(id_{DO})^{\sum_{i=1}^c \nu_i}, MPK\right) \cdot e\left(\prod_{i=1}^c H_4(\eta || SK_F || i)^{\nu_i} \cdot \prod_{j=1}^s A_j^{\sum_{i=1}^c \nu_i c_{f_i j}}, K_{dp}\right) \\ &= e\left(H_1(id_{DO})^{\sum_{i=1}^c \nu_i}, MPK\right) \cdot e\left(\prod_{i=1}^c H_4(\eta || SK_F || i)^{\nu_i} \cdot \prod_{j=1}^s A_j^{\mu_j}, K_{dp}\right) \end{aligned}$$

Theorem 2: Under the Random Oracle Model (ROM), the scheme in this chapter satisfies tag unforgeability for the adversary $\mathcal{A}_1$.

Proof: Provided with an case of the CDH problem (g, g^α, g^β) , $\mathcal{C}$ aims to compute $g^{\alpha\beta}$. The challenger $\mathcal{C}$ conducts system initialization to secure public parameters and designates the master key as $MSK = g^\alpha$. Challenger $\mathcal{C}$ transmits the master key MSK to $\mathcal{A}_1$ and interacts with $\mathcal{A}_1$ as shown below.

H_1 -Query: $\mathcal{A}_1$ is able to flexibly perform H_1 -Query dynamically against any picked ID'_0 . $\mathcal{C}$ sustains a list $L_{h_1} = (ID_0, h_1, D, T)$ to record H_1 -Query. If the picked ID'_0 is in L_{h_1} , $\mathcal{C}$ draws out the related tuple (ID'_0, h'_1, D', T') and replies with D' to $\mathcal{A}_1$. If it does not exist, $\mathcal{C}$ stochastically picks a value $h'_1 \in Z_q$ and flips a coin $T \in \{0, 1\}$. Assuming the probability that $T = 0$ is γ , and $1 - \gamma$ for $T = 1$. When $T = 0$ with probability γ , $\mathcal{C}$ calculates $D' = g^{h'_1}$. For $T = 1$, $D' = (g^\beta)^{h'_1}$. Then $\mathcal{C}$ transmits D' to $\mathcal{A}_1$ and appends the tuple (ID_0, h_1, D, T) in L_{h_1} .

Partial-Key-Query: $\mathcal{A}_1$ is able to perform an adaptive partial key query against any picked ID'_0 . $\mathcal{C}$ maintains a list $L_{SK} = (ID_0, S, SK_{ID_0}, PK_{ID_0})$ for the Partial-Key-Query. $\mathcal{C}$ scrutinizes ID'_0 is linked to L_{SK} . If it is not, $\mathcal{C}$ performs H_1 -Query for ID'_0 .

1) If $T' = 0$, $\mathcal{C}$ ceases.

2) If $T' = 1$ $\mathcal{C}$ computes $S' = D'^\beta = g^{\beta h'_1}$. Then $\mathcal{C}$ transmits S' to $\mathcal{A}_1$ and appends $(ID'_0, S', \perp)$ into L_{SK} .

Secret-Value-Query: $\mathcal{A}_1$ runs Secret-Value-Query dynamically for any picked ID'_0 . $\mathcal{C}$ scrutinizes ID'_0 and SK' are linked to L_{SK} . Unless, $\mathcal{C}$ performs H_1 -Query for ID'_0 . Subsequently, $\mathcal{C}$ scrutinizes if ID'_0 is in L_{SK} .

1) If $T' = 1$, $\mathcal{C}$ ceases.

2) If $T' = 1$, $\mathcal{C}$ chooses $x \in Z_q$ stochastically, and computes $SK_{ID'_0} = x$. Then $\mathcal{C}$ transmits $SK_{ID'_0}$ to $\mathcal{A}_1$ and appends $(ID_0, SK_{ID'_0}, PK_{ID'_0})$ into L_{SK} .

Public-Key-Query: $\mathcal{A}_1$ performs Public-Key-Query dynamically for any picked ID'_0 . $\mathcal{C}$ maintains a list $L_{SK} = (ID_0, S, SK_{ID_0}, PK_{ID_0})$. $\mathcal{C}$ examines whether ID'_0 and PK'_{ID_0} exist in L_{SK} . If not exist, $\mathcal{C}$ performs an H_1 -query. If PK'_{ID_0} doesn't exist, $\mathcal{C}$ performs a partial private key query and a secret value query, then $\mathcal{C}$ queries the value of PK'_{ID_0} in L_{SK} and transmits it to $\mathcal{A}_1$.

Public-Key-Replace: $\mathcal{A}_1$ dynamically runs Public-Key-Replace with picked $(ID'_0, PK'_{ID'_0})$. $\mathcal{C}$ examines whether ID'_0 exists in L_{SK} . If it exists, $\mathcal{C}$ updates the public key PK'_{ID_0} . If it does not exist, $\mathcal{C}$ increases $(ID'_0, PK'_{ID'_0})$ to L_{SK} .

H_2 -Query: $\mathcal{A}_1$ performs an H_2 -query adaptively for data f'_i . $\mathcal{C}$ maintains a list $L_3 = (f, h_2, Q)$. $\mathcal{C}$ searches the list L_{h_2} to determine whether f'_i exists. If L_{h_2} does not contain f'_i , $\mathcal{C}$ casually picks $h'_2 \in Z_p^*$, computes $Q = g^{h'_2}$ and inserts (f'_i, h'_2, Q) into L_{h_2} . $\mathcal{C}$ sends (f'_i, h'_2, Q) to $\mathcal{A}_1$.

Tag-Query: $\mathcal{A}_1$ executes Tag-Query with (ID'_0, f'_i) . $\mathcal{C}$ scrutinizes in L_{h_1} for ID'_0 . If $T' = 1$, $\mathcal{C}$ ceases. Or else, $\mathcal{C}$ obtains SK'_{ID_0} from L_{SK} , h'_2 from L_{h_2} . Then $\mathcal{C}$ produces the tag for f'_i and transmits the tag value to $\mathcal{A}_1$.

Forge: $\mathcal{A}_1$ outputs a tuple $O = (\sigma', SK'_{ID_0}, PK'_{ID_0}, f'_i, ID'_0)$, where σ' represents the forged tag.

Analysis: When the adversary $\mathcal{A}_1$ wins game 1, $\mathcal{C}$ is able to obtain the equation $e(\sigma_i, g) = e(H_1(id_{DO}), MPK) \cdot e(H_4(\eta \| SK_F \| i) \cdot \prod_{i=1}^n \prod_{j=1}^s A_j^{c_{ij}}, K_{dp})$. $\mathcal{C}$ obtains (ID'_0, h'_1, D', T') from L_{h_1} . When $T' = 0$, $\mathcal{C}$ ceases. Then $\mathcal{C}$ obtains $H_1(id'_{DO}) = g^{\beta h'_1}$ from L_{h_1} and $H_4(\eta \| SK_F \| i) = g^{h'_2}$ from L_{h_2} . By verifying the equation, $\mathcal{C}$ can be derived as $e(\sigma', g) = e(g^{\beta h'_2}, g^\alpha) \cdot e(g^{h'_2} \cdot \prod_{i=1}^n \prod_{j=1}^s A_j^{c_{ij}}, K_{dp})$. Then $\mathcal{C}$ can calculate $g^{\alpha\beta} = (\frac{\sigma'}{\prod_{i=1}^n \prod_{j=1}^s A_j^{c_{ij}} \cdot pk_{ID'_0}^{h'_2}})^{\frac{1}{h'_1}}$.

Here we denote the probability that $\mathcal{A}_1$ will obtain the correct result after querying H_1 -Query, Partial-Key-Query, Secret-Value-Query, H_2 -Query, Public-Key-Query, Public-Key-Replace, Replica-Query, and Tag-Query for up to $q_{H_1}, q_{SK}, q_{SV}, q_{H_2}, q_{PK}, q_{PKR}, q_R, q_T$. If $\mathcal{A}_1$ interacts perfectly with $\mathcal{C}$, $\mathcal{C}$ does not cease the algorithm during the query, then H_1 -Query, Secret-Value-Query, H_2 -Query, Public-Key-Query, Public-Key-Replace all execute effectively without extra conditions. In Partial-Key-Query, Replica-Query, and Tag-Query, $\mathcal{C}$ may cease the algorithm, and the probability of a perfect interaction between $\mathcal{C}$ and $\mathcal{A}_1$ exceeds $(1 - \gamma)^{q_{SK} q_R q_T}$. Therefore, $\mathcal{C}$ succeeds in obtaining the probability of $\epsilon' \geq \epsilon \gamma (1 - \gamma)^{q_{SK} q_R q_T} \geq \frac{\epsilon}{(q_{SK} + q_R + q_T) 2e}$. $\mathcal{C}$ can therefore tackle the CDH problem with the probability $\epsilon' \geq \frac{\epsilon}{(q_{SK} + q_R + q_T) 2e}$ is thus able to be used in time $t' \leq t + O(q_{H_1} + q_{SK} + q_{SV} + q_{H_2} + q_{PK} + q_{PKR} + q_R + q_T)$.

Theorem 3: In the ROM, the signature scheme in this chapter satisfies tag unforgeability against the adversary $\mathcal{A}_2$.

Proof: Provided with an case of the CDH problem (g, g^α, g^β) , $\mathcal{C}$ aims to compute $g^{\alpha\beta}$. $\mathcal{C}$ performs system initialization to gain parameters, generates the master key $s \in Z_q$ at random. $\mathcal{C}$ transmits the system master secret key and public parameters to $\mathcal{A}_2$, and interacts with $\mathcal{A}_2$.

H_1 -Query: $\mathcal{C}$ sustains a list $L_{h_1} = (ID_0, h_1, Q)$ to record H_1 -Query. If the picked ID'_0 is in L_{h_1} , $\mathcal{C}$ refines the related tuple (ID'_0, h'_1) to $\mathcal{A}_2$. If it does not exist, $\mathcal{C}$ stochastically alters a value $h'_1 \in Z_q$ and computes $Q' = g^{h'_1}$. Then $\mathcal{C}$ transmits Q' to $\mathcal{A}_2$ and appends the tuple (ID'_0, h'_1, Q') to L_{h_1} .

Secret-Value-Query: $\mathcal{A}_2$ performs Secret-Value-Query dynamically for any picked ID'_0 . $\mathcal{C}$ sustains a list $L_{h_2} = (ID_0, h_2, SK_{ID_0}, PK_{ID_0}, T)$ to record H_1 -Query. $\mathcal{C}$ scrutinizes ID'_0 and SK'_{ID_0} are linked to L_{SK} . If not, $\mathcal{C}$ fulfills H_1 -Query. If the private key SK'_{ID_0} does not exist, $\mathcal{C}$ randomly picks $x \in Z_q$ and installs $SK'_{ID_0} = x$. Subsequently, $\mathcal{C}$ scrutinizes if ID'_0 is in L_{SK} .

1) If $T' = 0$, $\mathcal{C}$ computes $PK'_{ID_0} = g^{SK'_{ID_0}}$.

2) If $T' = 1$, $\mathcal{C}$ computes $PK'_{ID_0} = g^{\alpha SK'_{ID_0}}$. Then $\mathcal{C}$ transmits (SK'_{ID_0}, PK'_{ID_0}) to $\mathcal{A}_2$ and appends them into L_{SK} .

Public-Key-Query: $\mathcal{A}_2$ performs Public-Key-Query dynamically for any picked ID'_0 . $\mathcal{C}$ maintains a list L_{SK} and checks whether ID'_0 and PK'_{ID_0} exist in L_{SK} . If not, $\mathcal{C}$ performs an H_1 -query. If PK'_{ID_0} doesn't exist, $\mathcal{C}$ fulfills a secret value query, then $\mathcal{C}$ queries the value of PK'_{ID_0} in L_{SK} and transmits it to $\mathcal{A}_2$.

H_2 -Query: $\mathcal{A}_2$ performs an H_2 -query adaptively for $\omega' \in Z_q$. $\mathcal{C}$ maintains a list $L_3 = (\omega, h_2, \eta)$. $\mathcal{C}$ searches the list

L_3 to determine whether ω' exists. If not, $\mathcal{C}$ randomly selects $h'_2 \in Z_q$, computes $\omega' = g^{\beta h'_2}$ and sends (ω', h'_2, η') to $\mathcal{A}_2$.

Tag-Query: $\mathcal{A}_2$ executes Tag-Query with (ω', f', ID'_0) . $\mathcal{C}$ scrutinizes if $T' = 0$ in L_{h_1} for ID'_0 . If $T' = 1$, $\mathcal{C}$ ceases. Or else, $\mathcal{C}$ obtains $SK'_{ID'_0}$ from L_{h_2} , and h'_2, η' from L_3 . Then $\mathcal{C}$ produces the tag for f'_i by fulfilling TagGen and transmits the tag value to $\mathcal{A}_2$.

Forge: $\mathcal{A}_2$ produces a tuple $O = (\sigma', \omega', f'_i, ID'_0)$, where σ' represents the forged tag.

Analysis: When $\mathcal{A}_2$ wins game 2, the $\mathcal{C}$ is able to obtain the equation $e(\sigma_i, g) = e(H_1(id_{DO}), MPK) \cdot e(H_4(\eta \| SK_F \| i)) \cdot \prod_{i=1}^n \prod_{j=1}^s A_j^{c_{ij}}, K_{dp}$. $\mathcal{C}$ obtains (ID'_0, h'_1, D', T') from L_{h_1} . When $T' = 1$, $\mathcal{C}$ ceases. Then $\mathcal{C}$ obtains $H_1(id'_{DO}) = g^{\beta h'_1}$ from L_{h_1} and $H_4(\eta \| SK_F \| i) = g^{h'_2}$ from L_{h_2} and $PK_{ID'_0} = (g^\alpha)^{x+h_2} \bmod q$ from q_{SV} . Based on the validation formula, $\mathcal{C}$ can get $e(\sigma', g) = e(g^{h'_1}, g^s) \cdot e(g^{\beta h'_2} \prod_{i=1}^n \prod_{j=1}^s A_j^{c_{ij}}, g^{\alpha(x+h'_2)})$. Then $\mathcal{C}$ can derive $g^{\alpha\beta} = (\sigma')^{sh'_1 h'_2 \prod_{i=1}^n \prod_{j=1}^s A_j^{c_{ij}} (x+h'_2)}$. Here we denote the probability that $\mathcal{A}_2$ will obtain the correct result within ϵ time S_t after querying H_1 -Query, Secret-Value-Query, H_2 -Query, Public-Key-Query, Replica-Query, and Tag-Query for up to $q_{H_1}, q_{SV}, q_{H_2}, q_{PK}, q_R, q_T$. If $\mathcal{A}_2$ interacts perfectly with $\mathcal{C}$, $\mathcal{C}$ does not cease the algorithm during the query, then H_1 -Query, Secret-Value-Query, H_2 -Query, and Public-Key-Query all execute successfully without additional conditions. In H_1 -Query, Replica-Query and Tag-Query, $\mathcal{C}$ may cease the algorithm, and the probability of a perfect interaction between $\mathcal{C}$ and $\mathcal{A}_2$ exceeds $(1 - \gamma)^{q_{H_1} q_R q_T}$. Therefore, $\mathcal{C}$ succeeds in obtaining the probability of $\epsilon' \geq \epsilon \gamma (1 - \gamma)^{q_{H_1} q_R q_T} \geq \frac{\epsilon}{(q_{H_1} + q_C + q_T)2e}$. $\mathcal{C}$ can thus tackle the CDH problem with the probability $\epsilon' \geq \frac{\epsilon}{(q_{H_1} + q_R + q_T)2e}$ is thus able to be used in time $t' \leq t + O(q_{H_1} + q_{SV} + q_{H_2} + q_{PK} + q_R + q_T)$.

Theorem 4: Under the DL problem, the scheme is secure against adversary $\mathcal{A}_3$. That is, if a malicious CSP fails to preserve data completely, the probability of it forging a valid integrity proof that passes DA verification with a non-negligible advantage is virtually zero. Any attempt to generate a valid proof for missing data blocks is computationally infeasible.

Proof: It is assumed that the hash function used is collision-resistant and the chosen signature algorithm is safe. While the CSP should generate a sound proof $P = \{\sigma, u_k\}$ based on intact data, to win Game 3, it must instead produce a falsified proof $P' = \{\sigma', u'_k\}$ derived from the corrupted data. Assuming that the $chal = (c, k_1, k_2)$ as a challenge message. If the forged certificate is permitted to be verified, the following formula needs to be met.

$$e(\sigma', g) = e\left(H_1(id_{DO})^{\sum_{i=1}^c v_i}, MPK\right) \cdot e\left(\prod_{i=1}^c H_4(\eta \| SK_F \| i)^{v_i} \cdot \prod_{j=1}^s A_j^{u'_j}, K_{dp}\right)$$

Since valid proofs can be verified, we can derive the formula:

$$e(\sigma, g) = e\left(H_1(id_{DO})^{\sum_{i=1}^c v_i}, MPK\right) \cdot$$

$$e\left(\prod_{i=1}^c H_4(\eta \| SK_F \| i)^{v_i} \cdot \prod_{j=1}^s A_j^{u_j}, K_{dp}\right)$$

Compared with the above two equations, at least one of the formulas is $u_{j'} = u_j$. Also since individual tags cannot be forged, there must be $\sigma' = \sigma$, and obtains $\sum_{j=1}^s a_j u_j = \sum_{j=1}^s a_j u_{j'}$, namely $\sum_{j=1}^s a_j (u_j - u_{j'}) = 0$. Assuming that there are λ distinct $u_j, u_j \neq u_{j'}$, the count of tuples $(\alpha_1, \dots, \alpha_s)$ satisfying the attribute is at most $q^{\lambda-1}$. Since $(\alpha_1, \dots, \alpha_s)$ is unknown for CSP, the probability of the equation $\sum_{j=1}^s a_j (u_j - u_{j'}) = 0$ is less $q^{\lambda-1}/q^s \leq q^{\lambda-1}/q^\lambda = 1/q$, which is negligible. Therefore, without storing the complete data, cloud storage services cannot pass data integrity verification by fabricating audit proof.

Theorem 5: Assuming that DBDH holds then, this scheme has indistinguishability under chosen-plaintext attack.

Proof: The challenger $\mathcal{C}$ can choose a set of attribute in the non-access policy γ and generate public parameters $para = \{G, G_T, q, e, g, g^\beta, e(g, g)^\alpha, H_1, H_2, H_3, H_4, \pi, prf, prp, MPK\}$, where G, G_T is the cyclic group of two large prime numbers, g is the generating element of G . Suppose the challenger flips a coin and $\mathcal{C}$ is invisible, then if $\mu = 0$ then the challenger sets $(\omega, \lambda, \sigma, \delta) = (g^a, g^b, g^c, e(g, g)^{abc})$, otherwise sets $(\omega, \lambda, \sigma, \delta) = (g^a, g^b, g^c, e(g, g)^\delta)$. $\mathcal{A}_4$ uses the access policy γ to interact with $\mathcal{C}$. $\mathcal{C}$ gives the generated public parameter $para$ to $\mathcal{A}_4$. $\mathcal{C}$ write down the following ranges. $e(g, g)^y = e(\omega, \lambda) = e(g, g)^{ab}$, and by selecting $\beta' \in Z_q$, record this $g^{\beta'} = g^\beta$.

Query phase I: When the adversary makes a hash query, $\mathcal{C}$ checks its hash list. If the queried value exists, $\mathcal{C}$ returns the stored hash. Otherwise, $\mathcal{C}$ generates random $x_i, y_i, z_i \in Z_q$ and records them accordingly.

$$H_1(h_i) = \begin{cases} g^{x_i}, & \text{if } h_i = id_{DUi} \\ g^{y_i}, & \text{if } h_i = j_i \\ g^{z_i}, & \text{if } h_i = att(\gamma)_i \end{cases}$$

$\mathcal{C}$ saves $H_1(h_i)$ in a hash list, $\mathcal{A}_4$ randomly selects the set of attributes to request an identity key from $\mathcal{C}$. $\mathcal{C}$ randomly selects $\epsilon'_1, \epsilon', r'_j \in Z_q$, set $\epsilon_1 = \epsilon_i$, $\epsilon_2 = a\epsilon'_2/(ab + \omega_i)$, $r_j = (b + r'_j)/\epsilon_1$ and calculation:

$$\begin{aligned} D_1 &= (g^\alpha \cdot H_1(id_{DUi}))^{\epsilon_2 \beta} \\ &= (g^{ab} \cdot g^{x_i})^{a\epsilon_i/(ab + x_i)\beta'} \\ &= g^{(a\epsilon'_i)\beta'} = \omega^{\epsilon_2/\beta} \\ D_j &= H_1(id_{DUj})^{\epsilon_1} \cdot H_1(j_i)^{\epsilon_i r_j} \\ &= (g^{x_i})^{\epsilon_1} \cdot (g^{y_i})^{b+r_j} \\ &= g^{x_i \epsilon'_1} \cdot g^{y_i r_j} \cdot \lambda^{y_i} \end{aligned}$$

Challenge phase: $\mathcal{A}_4$ randomly picks two messages of equal length L_0, L_1 and passes them to $\mathcal{C}$. $\mathcal{C}$ casually picks $m \in (0, 1)$ to encrypt L_m and acquires the ciphertext FC :

$$FC = \begin{pmatrix} C_{L_0} = L_0 \cdot e(g, g)^{abc} \\ C_1 = \sigma^\lambda, \forall y = \gamma \\ C_2 = \sigma \cdot g^{\epsilon_i} \\ C_3 = \sigma^{z_i} \cdot g^{z_i \epsilon_1} \end{pmatrix}$$

If $\mu = 0$, then $\delta = e(g, g)^{abc}$, and setting $T_1 = c$, we can obtain: $C_{L_m} = L_m \cdot e(g, g)^{abc}$, $C_i = g^{\beta T_1} = g^{bc} = \sigma^\delta$. Then, the leafnode is $c + \epsilon_i (\epsilon_i \in Z_q)$. Hence, the file is encrypted randomly against L_m . Otherwise, $\delta = e(g, g)^\delta$, we can get $c_i = C_i \cdot e(g, g)^\delta$.

Query stage 2: $\mathcal{C}$ repeats the operation as in the query stage 1.

Guessing stage: $\mathcal{A}_4$ outputs a guess b' for b . If $b = b'$, $\mathcal{C}$ outputs $v' = 0$, indicating a valid DBDH tuple. Otherwise, $\mathcal{C}$ outputs $v' = 1$, indicating a random tuple. If $v' = 0$, then adversary $\mathcal{A}_4$ has obtained the encryption of L_m , the advantage of $\mathcal{A}_4$ is ξ . Thus, $\Pr[x \neq x' \mid v = 0] = 1/2 + \xi$, when $e' = e$, $\mathcal{C}$ guess $v' = 0$ then we can get $\Pr[v = v' \mid v = 0] = 1/2 + \xi$. If $v' = 1$, then the adversary $\mathcal{A}_4$ does not know b , and thus $\Pr[x \neq x' \mid v = 1] = 1/2$. Now the advantage is $1/2 \Pr[v = v' \mid v = 1] + 1/2 \Pr[v = v' \mid v = 0] - 1/2 = \xi/2$.

If the adversary's advantage in probabilistic poly-time is ξ , then the $\xi/2$ advantage for the DBDH problem is non-negligible.

Theorem 6: Under the ROM, if the CDH problem is hard, this scheme resists collusive attacks by DA and CSP.

Proof: Provided with an instance of the CDH problem (g, g^α, g^β) , $\mathcal{C}$ aims to compute $g^{\alpha\beta}$. The challenger $\mathcal{C}$ conducts system initialization to secure public parameters and designates the master secret key as $MSK = g^\alpha$. Challenger $\mathcal{C}$ transmits the system master secret key MSK to $\mathcal{A}_5$ and interacts with $\mathcal{A}_5$ as shown below.

Query phase: When $\mathcal{A}_5$ queries $H_1(id'_{DO})$, $\mathcal{C}$ randomly selects $h_1 \in Z_q$, and with probability $1 - \delta$ sets $H_1(id'_{DO}) = (g^\beta)^{h_1}$. $\mathcal{C}$ stores the tuple $(ID'_0, h'_1, H_1(id'_{DO}))$ in the list L_{H_1} . When $\mathcal{A}_5$ queries the private key of DO, $\mathcal{C}$ first executes an H_1 -Query. If $H_1(id'_{DO})$ is set to $(g^\beta)^{h'_1}$, then $\mathcal{C}$ aborts the simulation. Otherwise, $\mathcal{C}$ can generate and return the private key normally according to the scheme.

Challenge phase: $\mathcal{A}_5$ submits the challenge file F' . $\mathcal{C}$ selects one data block f'_i from F' as the challenge target. $\mathcal{C}$ simulates the tag generation process for F' . For the challenge block f'_i , $\mathcal{C}$ programs the H_4 -query such that $H_4(\eta' \| SK'_F \| i) = g^c$, where c is randomly chosen by $\mathcal{C}$. $\mathcal{C}$ calculates and returns the tag σ'_i . According to the TagGen algorithm of the scheme, a valid tag should satisfy the following structure:

$$\sigma'_i = (H_1(id'_{DO})^\alpha) \cdot \left(g^c \cdot g^{\sum_{j=1}^s a_j c_{ij}} \right)^{\alpha_{DO}}$$

Here, $\mathcal{C}$ can know or set $H_1(id'_{DO}) = g^\beta$ and $K_{dp} = g^{\alpha_{DO}}$ through previous queries. $\mathcal{C}$ subsequently tampers with this data block.

Forge: According to the assumption, $\mathcal{A}_5$, leveraging its collusion with the CSP, is able to generate a forged proof $Proof'$ for the tampered data, which contains a forged tag aggregation value σ' that can pass verification. The verification equation requires:

$$e(\sigma', g) = e(H_1(id'_{DO}), MPK) \cdot e\left(\prod H_4(\cdot)^{v_i} \cdot \prod A_j^{\mu_j}, K_{dp}\right)$$

Since $\mathcal{A}_5$ does not possess the DO's private key α_{DO} , to pass verification, it must construct a σ' that satisfies the

equation without knowing α and α_{DO} . $\mathcal{C}$ extracts σ' from the $Proof'$ submitted by $\mathcal{A}_5$. $\mathcal{C}$ can represent the scheme $g^{\alpha\beta}$ to the CDH problem.

Analysis: If $\mathcal{A}_5$ wins the game with advantage ϵ , then the advantage of $\mathcal{C}$ successfully solving the CDH problem is $\epsilon' \geq \frac{\epsilon}{(q_{H_1} \cdot q_T)}$, where q_{H_1} and q_T are the upper bounds on the number of H_1 -queries and tag queries, respectively. Since ϵ is non-negligible and both q_{H_1} and q_T are polynomial, ϵ' is also non-negligible. There does not exist a polynomial-time adversary $\mathcal{A}_5$ that wins the security game against the collusion attack between DA and CSP with non-negligible advantage. Therefore, our scheme is secure.

Theorem 7: Even if the DA obtains all audit interaction information, it is computationally impossible to derive any raw data content belonging to the DO from this information.

Firstly, to prevent malicious DAs from harming DO's rights or stealing data privacy, the system employs a strict identity authentication mechanism to ensure only authorized DAs can participate in audits. DAs must use their private key k_{das} to verify the proof-of-possession I' and C' sent by the CSP. This process enables DA identity verification, tracking, and accountability. This non-repudiation feature encourages DAs to exercise greater caution during verification, effectively preventing data leaks.

Secondly, DA, driven by curiosity about the data content, attempts to infer user data through proofs sent via CSP. However, it can only obtain the values of I' and C' . Since $I' = e(H_{DA} K_{tp})$, $C' = H_{DA} C$, and given the computational difficulty of discrete logarithms and the challenge of inverting bilinear pairings, DA cannot derive any information related to f_i .

Theorem 8: The scheme satisfies the tag deduplication requirement, ensuring the uniqueness of each tag.

In the scheme, the DO uses the file private key SK_F to generate an authenticator, calculated as $\sigma_i = K_{dos} \cdot (H_4(\eta \| SK_F \| i) \cdot g^{\sum_{j=1}^s a_j c_{ij}})^{\alpha_{DO}}$. SK_F can be derived from $\eta = H_3(k_l \| kc_i)$ and its extension key kc_i . Since kc_i is a convergent key, any uploader possessing the same file can recover the identical file private key, thereby generating consistent authenticators. Consequently, different owners with identical file content will obtain the same authenticator. The CSP only needs to store a single copy of the authenticator, effectively meeting the requirement for tag deduplication.

Theorem 9: The scheme provides proof of data ownership.

In our scheme, the CSP sends a challenge (c, k_1, k_2) to user DO_{su} . DO_{su} must compute the proof $PoW.Proof$ based on the local file F , which includes the ciphertext C_i encrypted using the convergence key kc_i . The CSP verifies this proof by comparing it against the stored copy of the authenticator, thereby confirming DO_{su} 's full ownership of file F . After ownership verification, if DO_{su} requires data access, they must still initiate an access request to the CSP. Decryption is permitted only when the user's attributes satisfy the access policy γ embedded within the ciphertext FC . This enables retrieval of the ciphertext FC sent by the CSP, ultimately leading to the decryption and restoration of the plaintext F . During the data integrity audit phase, DA verifies proofs

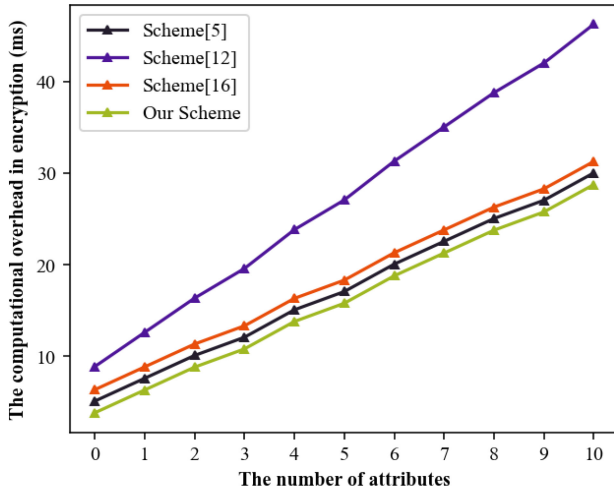


Fig. 3. Computational cost of encryption phase.

based on the tag σ_i . This authenticator is generated by the initial uploader and bound to their private key K_{dos} and secret value α_{DO} . The verification process relies on the DO's public key K_{dp} , ensuring this scheme fully satisfies the requirements for data ownership proof.

Theorem 10: Under the ROM and the DBDH assumption, the hybrid encryption scheme simultaneously achieves secure RCE deduplication and CP-ABE fine-grained access control.

Data-layer encryption employs the form $C_i = ENC_1(kc_i, f_i)$, where $kc_i = \pi(ks, h_i)$, $h_i = H_2(f_i)$. The collision resistance of hash function H_2 ensures the probability of different plaintexts mapping to the same kc_i is negligible, guaranteeing identical plaintext f_i generates identical ciphertext C_i . Simultaneously, the semantically secure symmetric encryption algorithm ENC_1 guarantees C_i does not reveal any information about f_i .

At the policy layer, kc_i is encapsulated using CP-ABE and embedded within the access policy γ , generating a ciphertext structure containing components such as c_1, c_{y_0}, c'_{y_1} . CP-ABE based on the DBDH assumption satisfies IND-CPA security, ensuring decryption only by users whose attributes satisfy the policy.

Deduplication relies solely on the consistency of data-layer ciphers C_i , without leaking keys or attribute association information. Attackers cannot derive kc_i from C_i nor reconstruct key parameters from policy-layer components, thereby preserving fine-grained access control and security while supporting deduplication.

In summary, the hybrid encryption structure guarantees fine-grained access control security while supporting ciphertext deduplication. The proof demonstrates the compatibility of deduplication and access control based on the ROM and the DBDH assumption.

VII. PERFORMANCE ANALYSIS

A. Functional Comparison

We analyse the functionality of our scheme and compare the functionality with various other related schemes in Table II, such as certificate-less signature, CP-ABE, designate auditor,

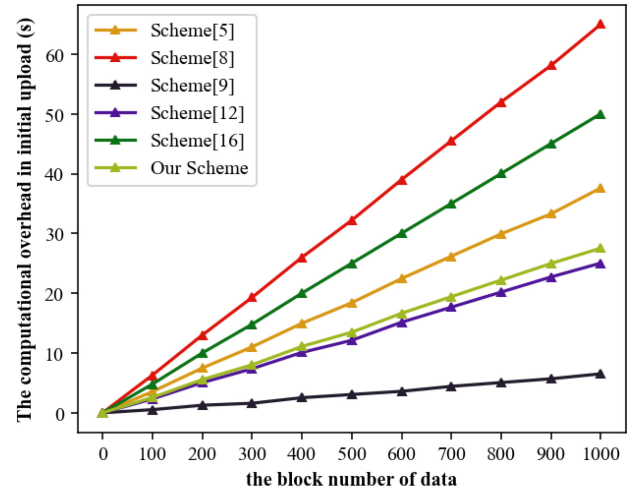


Fig. 4. Computational cost of initial upload phase.

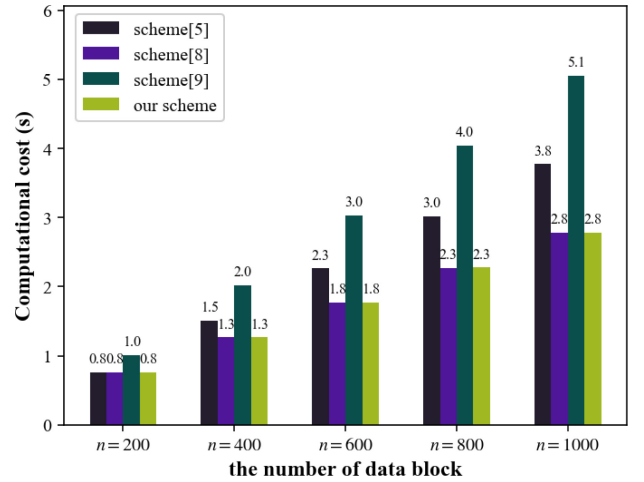


Fig. 5. Computational cost of data auditing phase.

data integrity, and data deduplication. We can see that the other compared schemes implement only some of these features, whereas our scheme implements all of them, providing a more comprehensive scheme. $\times$ indicates that the function is not implemented in the program. $\checkmark$ indicates that the function is implemented.

B. Theoretical Analysis

Here, we theoretically analyze the computational and communication overhead in our scheme since they are the most resource-intensive phases. Other operations in Z_q , such as hashing and addition, have negligible costs and can therefore be omitted. In subsequent analyses, since the computational cost of CP-ABE operations increases linearly with the number of attributes, N_C can also serve as a metric for policy complexity.

1) Computational Overhead: The scheme's computational cost consists of encryption and decryption of CP-ABE, initial uploads, and subsequent uploads (*PoW.Gen* and *PoW.Ver*), *Proof.Gen* and *Proof.Verify* between CSP and DA. For specific data, please refer to Table III.

TABLE II
COMPARISONS OF FUNCTIONALITY

Scheme	Certificate-less	CP-ABE	Designate Auditor	Data Integrity	Data Deduplication
Scheme [5]	×	✓	×	✓	×
Scheme [8]	×	×	×	✓	✓
Scheme [9]	×	×	×	✓	✓
Scheme [12]	×	✓	×	✓	✓
Scheme [15]	×	✓	×	✓	✓
Scheme [16]	×	✓	×	×	×
Scheme [26]	✓	×	✓	✓	×
Our Scheme	✓	✓	✓	✓	✓

TABLE III
COMPUTATIONAL OVERHEAD COMPARISON

	Scheme [5]	Scheme [8]	Scheme [9]	Scheme [12]	Scheme [16]	Our Scheme
Encryption	$(2 + 2N_C)T_E + 2T_M$	/	/	$(4 + 5N_C)T_E + T_M$	$(3 + 2N_C)T_E + 2T_M$	$(2 + 2N_C)T_E + T_M$
Initial upload	$2nS_{DU}T_E + nS_{DU}T_M$	$2nT_E + n(s + 1)T_M$	$2nT_E + 3nT_M$	$nS_{DU}T_E + nS_{DU}T_M$	$2nN_C T_E + 2nS_{DU}T_M$	$n(1 + S_{DU})T_E + n(1 + S_{DU})T_M$
Pow.Gen	/	$cT_E + cT_M$	$cT_E + cT_M$	/	/	$cT_E + cT_M$
Pow.Ver	/	$(c + s)T_E + 2T_P + (c + s)T_M$	$2cT_M + 2T_P + (2c + 1)T_E$	/	/	$(c + s)T_E + 3T_P + (c + s)T_M$
Proof-Gen	$cT_E + cT_M$	$cT_E + cT_M$	$cT_E + cT_M$	/	/	$cT_E + cT_M$
Proof-Verify	$2cT_E + cT_M + 2T_P$	$(c + s)T_E + 2T_P + (c + s)T_M$	$2cT_M + 2T_P + (2c + 1)T_E$	/	/	$(c + s)T_E + 3T_P + (c + s)T_M$
Decryption	$(2 + 2S_{DU})T_E + 2N_S T_M$	/	/	$(1 + 3S_{DU})T_E + N_S T_M$	$(1 + 2S_{DU})T_E + (1 + 2N_S)T_M$	$(1 + 2S_{DU})T_E + N_S T_M$

Our scheme has a computational cost of $(2 + 2N_C)T_E + T_M$ in the encryption stage. In this context, our scheme is more effective compared to scheme [5], scheme [12], and scheme [16]. The scheme [8] and scheme [9] do not enable outsourced data encryption.

In the initial upload phase, the computational overhead of our scheme is primarily attributed to the tags generation operation, which is quantized as $n(1 + S_{DU})T_E + n(1 + S_{DU})T_M$. In the tags generation process, we adopt a certificate-less cryptosystem, which effectively avoids the risk of key escrow and eliminates certificate management. In addition, our scheme, like scheme [5], scheme [12], and scheme [16], has a tag generation process that is closely related to the user's attributes. In contrast, scheme [8] and scheme [9]

do not implement the access control of unauthorized users, and the tag generation doesn't depend on the user's attributes.

In the subsequent upload phase, since scheme [5], scheme [12], and scheme [16] do not implement this mechanism, this paper selects only scheme [8] and scheme [9], which similarly support PoW verification for performance evaluation comparison. During this phase, subsequent uploaders must execute an additional PoW protocol process, comprising two steps: *PoW.Gen* and *PoW.Ver*. The CSP is responsible for assisting users in generating file tags bound to public keys, verifying the association between ciphertext and tags, and completing ownership verification through interaction with the uploader. During the *PoW.Gen* phase, the computational overhead of this scheme is $cT_E + cT_M$; during the *PoW.Ver*

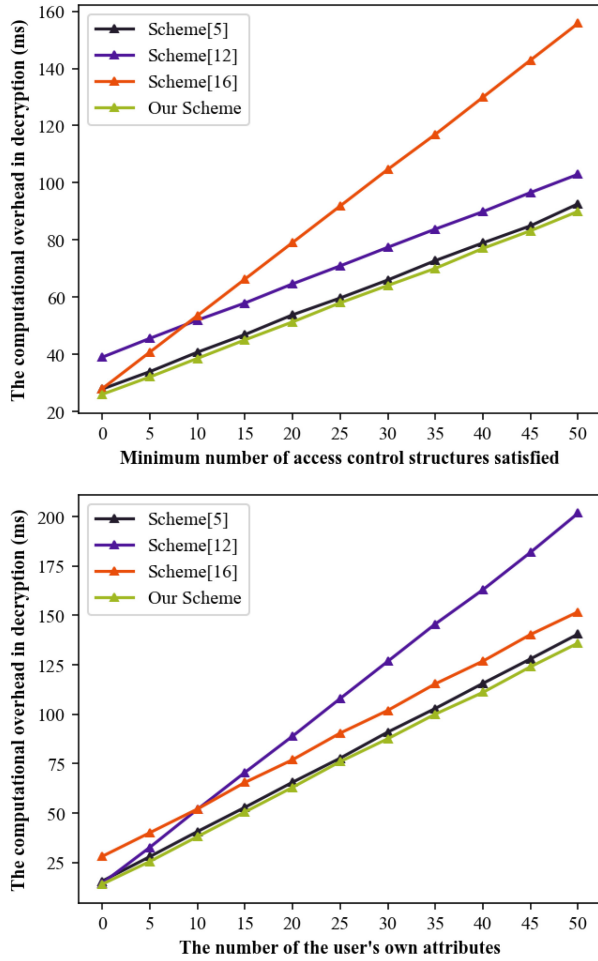


Fig. 6. Computational cost of decryption phase.

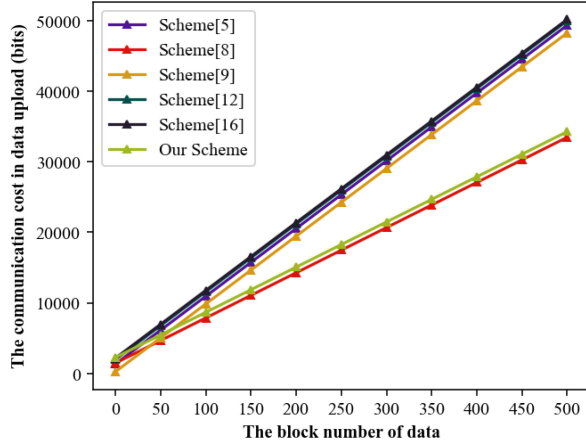


Fig. 7. Communication cost of data upload phase.

phase, the computational overhead is $(c+s)T_E + (c+s)T_M + 3T_P$.

The computational overhead of the data integrity audit is calculated in a similar way to the overhead of PoW. We contrast our scheme with scheme [5], scheme [8], and scheme [9], where the cost is the same as the overhead of the subsequent upload phase, the overhead of *Proof.Gen* phase is $cT_E + cT_M$, and the cost of the *Proof.Verify* phase is $(c+s)T_E + (c+s)T_M + 3T_P$.

TABLE IV
COMMUNICATION COST COMPARISON

	Data upload	Data auditing
Scheme [5]	$(1 + 2N_C + n) G + n Z_q $	$ G + 4 Z_q $
Scheme [8]	$(n + 14) G + (s + 8)Z_q$	$ G + 4 Z_q $
Scheme [9]	$(n + 1) G + (n + 6) Z_q $	$ G + 4 Z_q $
Scheme [12]	$(3N_C + 1 + n) G + n Z_q $	/
Scheme [16]	$(3N_C + 4 + n) G + n Z_q $	/
Our Scheme	$(n + 9 + 2N_C) G + (s + 3) Z_q $	$ G + 4 Z_q $

In the decryption phase, the cost of our scheme is $(1 + 2S_{DU})T_E + N_S T_M$. The cost of scheme [5] is $(2 + 2S_{DU})T_E + 2N_S T_M$, the overhead cost of scheme [12] is $(1 + 3S_{DU})T_E + N_S T_M$, the cost of scheme [16] is $(1 + 2S_{DU})T_E + T_E(1 + 2N_S)T_M$. Our scheme does not require many decryption operations during the decryption process.

2) *Communication Cost*: We mainly contrast the communication cost in the schemes in terms of data upload phase and data auditing phase. For specific data, please refer to Table IV.

In the data upload phase, our scheme primarily consists of two stages: initial upload and subsequent upload. Throughout both initial and subsequent uploads, communication overhead primarily stems from interactions between users and clouds. When uploading n data blocks, the user must perform duplicate data detection, generate a public file tag, and upload the authentication tag and ciphertext. The total communication cost is $(n + 9 + 2N_C)|G| + (s + 3)|Z_q|$. Similar to our approach, Schemes [8] and scheme [9] both implement data deduplication with initial and subsequent upload phases. Specifically, Scheme [8] incurs a communication cost of $(n + 14)|G| + (s + 8)|Z_q|$, while Scheme [9] incurs $(n + 1)|G| + (n + 6)|Z_q|$. In contrast, Schemes [5], scheme [12], and scheme [16] lack deduplication capabilities. Therefore, we primarily focus on their communication costs during data uploads. The communication cost for Scheme [5] is $(1 + 2N_C + n)|G| + n|Z_q|$, Scheme [12] is $(3N_C + n + 1)|G| + n|Z_q|$, and Scheme [16] is $(3N_C + n + 4)|G| + n|Z_q|$.

During data integrity auditing, our communication overhead is mainly between the TPA and the CSP, which is a relatively small percentage of the process. The scheme [12] and scheme [16] fail to support data integrity audits. The communication cost of our scheme is the same as the communication cost of several other schemes, which is $|G| + 4|Z_q|$.

C. Experimental Result

In order to implement our scheme and the related ICP. The experimental computer was equipped with the Ubuntu 22.04.2 LTS Linux operating system on the server, which has an Intel (R) Xeon (R) Gold 6133 CPU @ 2.50GHz and 8.0GB of RAM, to utilize the PBC (Pairing-based Cryptography) library.

1) *Computational Overhead*: According to Fig. 3, We compared with scheme [5], scheme [12], scheme [16] and our scheme during the encryption phase, whose computational overhead in the encryption phase is related to the count of attributes, but it is observable that the computational overhead of our scheme is not as costly as the other three schemes, which has some advantages.

According to Fig. 4, we compared the computational overheads of different schemes during the initial upload phase. We set $s = 50$, $N_C = 10$, $S_{DU} = 10$. Our scheme has a significantly lower computational overhead than the scheme [5], scheme [8], and scheme [16], and the comparison of their overheads becomes more obvious as the number of data blocks increases. In addition, compared to Scheme [8], our scheme is independent of the number of sectors.

In our scheme, in the subsequent upload part of the data, as well as the data auditing process, it is necessary to use the PoW to prove, so the scheme overhead is calculated in a similar way, and we mainly show the comparison of the results of the data integrity auditing process, as shown in Fig. 5. we set $s = 100$, comparing the communication overhead in the case of $n = 200, 400, 600, 800, 1000$ respectively. Among the comparison schemes, only scheme [8], scheme [9] embody all full proofs. By comparing, we can see that our scheme is affected by the number of sectors, therefore, the computation overhead is closely tied to the number of sectors, but under certain conditions, our scheme still has some advantages.

Fig. 6 compares the computational overhead of different schemes in the decryption phase. We analyze the computational overhead when $N_S = 10$, S_{DU} goes from 0 to 50 in intervals of 5, and when $S_{DU} = 10$, N_S goes from 0 to 50 in intervals of 5, respectively. It can be seen that our overhead is lower than scheme [5], scheme [12], and scheme [16], so our scheme is more advantageous.

2) *Communication Cost*: In Fig. 7, we contrast the communication overheads of different schemes in the data uploading phase. The communication overhead in the auditing phase can be ignored since it is less. We compare and analyse the experimental results only from the data upload phase. We set $N_C = 10$, $s = 10$, n goes from 0 to 500 in intervals of 50. As is evident, the communication overhead of our scheme is lower when compared to Scheme [5], Scheme [9], Scheme [12], and Scheme [16]. The computation overhead is slightly higher than that of scheme [8].

VIII. CONCLUSION

In the paper, we propose an integrity auditing scheme built on attribute encryption and deduplication, which is built on the model of attribute encryption and convergent key encryption scheme that enables data sharing. We can substantively realize fine-grained access control through CP-ABE technique to prevent malicious users from stealing private information for privacy protection. Initial and subsequent uploads of data are achieved using convergent key encryption and proof of ownership of data, respectively. In addition, our scheme requires the CSP to appoint an auditor for auditing. Meanwhile, we solve

the complex key escrow problem based on a certificate-less cryptosystem. Finally, based on CDH and DBDH assumptions, our scheme is demonstrably secure.

In the future, our goal is to break the limitations of experimental environments by deploying schemes in distributed cloud environments or test platforms to evaluate their scalability for large-scale data. At the same time, by integrating CP-ABE with blockchain technology, access policies will be encoded into smart contracts to guarantee the immutability and auditability of access logs, as well as the automation of policy enforcement.

REFERENCES

- [1] H. Tian, F. Nan, C.-C. Chang, Y. Huang, J. Lu, and Y. Du, "Privacy-preserving public auditing for secure data storage in fog-to-cloud computing," *J. Netw. Comput. Appl.*, vol. 127, pp. 59–69, Feb. 2019.
- [2] J. Zhang, Y. Yang, Y. Chen, J. Chen, and Q. Zhang, "A general framework to design secure cloud storage protocol using homomorphic encryption scheme," *Comput. Netw.*, vol. 129, pp. 37–50, Dec. 2017.
- [3] A. Goel, C. Prabha, M. Malik, and P. Sharma, "Security concerns and data breaches for data deduplication techniques in cloud storage: A brief meta-analysis," *Int. J. Saf. Secur. Eng.*, vol. 14, no. 2, pp. 435–446, 2024.
- [4] T. Jiang, X. Chen, Q. Wu, J. Ma, W. Susilo, and W. Lou, "Secure and efficient cloud data deduplication with randomized tag," *IEEE Trans. Inf. Forensics Security*, vol. 12, pp. 532–543, 2016.
- [5] Y. Yao, J. Chang, and A. Zhang, "Efficient data sharing scheme with fine-grained access control and integrity auditing in terminal-edge-cloud network," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 26944–26954, Aug. 2024.
- [6] X. Zheng, W. Shen, Y. Su, and Y. Gao, "DIADD: Secure deduplication and efficient data integrity auditing with data dynamics for cloud storage," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 299–316, Feb. 2025.
- [7] Z. Hua, Y. Yao, M. Song, Y. Zheng, Y. Zhang, and C. Wang, "Blockchain-assisted secure deduplication for large-scale cloud storage service," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 821–835, May/Jun. 2024.
- [8] Y. Miao, K. Gai, L. Zhu, K.-K. R. Choo, and J. Vaidya, "Blockchain-based shared data integrity auditing and deduplication," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3688–3703, Jul./Aug. 2024.
- [9] G. Tian et al., "Blockchain-based secure deduplication and shared auditing in decentralized storage," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 3941–3954, Nov./Dec. 2022.
- [10] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Proc. 32nd Annu. Int. Conf. Theory Appl. Cryptogr. Tech.*, 2013, pp. 296–312.
- [11] J. Yuan and S. Yu, "Secure and constant cost public cloud storage auditing with deduplication," in *Proc. IEEE Conf. Commun. Netw. Security (CNS)*, 2013, pp. 145–153.
- [12] T. Zhou, Z. Tang, S. Zeng, and M. Shao, "Deduplication-enabled CP-ABE with revocation," *Peer Peer Netw. Appl.*, vol. 18, no. 2, p. 9, 2025.
- [13] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 89–98.
- [14] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *Proc. IEEE Symp. Security Privacy (SP)*, 2007, pp. 321–334.
- [15] J. Ma, M. Wang, J. Xiong, and Y. Hu, "CP-ABE-based secure and verifiable data deletion in cloud," *Secur. Commun. Netw.*, vol. 2021, no. 1, 2021, Art. no. 8855341.
- [16] S. Fugkeaw, L. Wirz, and L. Hak, "Secure and lightweight blockchain-enabled access control for fog-assisted IoT cloud based electronic medical records sharing," *IEEE Access*, vol. 11, pp. 62998–63012, 2023.
- [17] H. Shacham and B. Waters, "Compact proofs of retrievability," *J. Cryptol.*, vol. 26, no. 3, pp. 442–483, 2013.

- [18] S. Naik and K. P. D. Subramanya, "A cryptographic technique for cloud computing's secure data storage for third-party public auditing schemes," *AIP Conf. Proc.*, vol. 3278, no. 1, 2025, Art. no. 20023.
- [19] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.
- [20] I. K. Sahu and M. J. Nene, "Identity-based integrity verification (IBIV) protocol for cloud data storage," in *Proc. Int. Conf. Adv. Electr. Comput., Commun. Sustain. Technol. (ICAECT)*, 2021, pp. 1–6.
- [21] R. Rabaninejad, M. R. Asaar, M. A. Attari, and M. R. Aref, "An identity-based online/offline secure cloud storage auditing scheme," *Clust. Comput.*, vol. 23, no. 2, pp. 1455–1468, 2020.
- [22] L. Huang, J. Zhou, G. Zhang, and M. Zhang, "Certificateless public verification for data storage and sharing in the cloud," *Chin. J. Electron.*, vol. 29, no. 4, pp. 639–647, 2020.
- [23] X. Guan, J. Chang, and W. Zhang, "Secure data sharing scheme with privacy-preserving and certificateless integrity auditing in cloud storage," *Comput. Commun.*, vol. 224, pp. 285–301, Aug. 2024.
- [24] Y. Yu et al., "Enhanced privacy of a remote data integrity-checking protocol for secure cloud storage," *Int. J. Inf. Secur.*, vol. 14, pp. 307–318, Aug. 2015.
- [25] Y. Ren, J. Xu, J. Wang, and J.-U. Kim, "Designated-verifier provable data possession in public cloud storage," *Int. J. Secur. Appl.*, vol. 7, no. 6, pp. 11–20, 2013.
- [26] R. Li, X. A. Wang, H. Yang, K. Niu, D. Tang, and X. Yang, "Efficient certificateless public integrity auditing of cloud data with designated verifier for batch audit," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 10, pp. 8079–8089, 2022.
- [27] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. 22nd Int. Conf. Distrib. Comput. Syst.*, 2002, pp. 617–624.
- [28] Y. Zhang, C. Xu, N. Cheng, and X. Shen, "Secure password-protected encryption key for deduplicated cloud storage systems," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2789–2806, Jul./Aug. 2022.
- [29] Y. Zheng, X. Yuan, X. Wang, J. Jiang, C. Wang, and X. Gui, "Toward encrypted cloud media center with secure deduplication," *IEEE Trans. Multimedia*, vol. 19, no. 2, pp. 251–265, Feb. 2017.
- [30] Y. Xu, C. Zhang, G. Wang, Z. Qin, and Q. Zeng, "A blockchain-enabled deduplicatable data auditing mechanism for network storage services," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1421–1432, Jul.–Sep. 2021.
- [31] J. Bai, J. Yu, and X. Gao, "Secure auditing and deduplication for encrypted cloud data supporting ownership modification," *Soft Comput.*, vol. 24, no. 16, pp. 12197–12214, 2020.
- [32] X. Liu, W. Sun, W. Lou, Q. Pei, and Y. Zhang, "One-tag checker: Message-locked integrity auditing on encrypted cloud deduplication storage," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2017, pp. 1–9.
- [33] X. Gao et al., "Achieving low-entropy secure cloud data auditing with file and authenticator deduplication," *Inf. Sci.*, vol. 546, pp. 177–191, Feb. 2021.
- [34] X. Zhang, Y. Su, and J. Qin, "A dynamic searchable symmetric encryption scheme for multiuser with forward and backward security," *Secur. Commun. Netw.*, vol. 2020, no. 1, 2020, Art. no. 8893016.
- [35] B. Waters, "Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization," in *Proc. 14th Int. Conf. Pract. Theory Workshop Public Key Cryptogr.*, 2011, pp. 53–70.
- [36] Y. Yu, Y. Li, B. Yang, W. Susilo, G. Yang, and J. Bai, "Attribute-based cloud data integrity auditing for secure outsourced storage," *IEEE Trans. Emerg. Topics Comput.*, vol. 8, no. 2, pp. 377–390, Apr.–Jun. 2020.
- [37] S. Chen, J. Li, Y. Zhang, and J. Han, "Efficient revocable attribute-based encryption with verifiable data integrity," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10441–10451, Mar. 2024.
- [38] P. Goswami, N. Faujdar, S. Debnath, A. K. Khan, and G. Singh, "ZSS signature-based audit message verification process for cloud data integrity," *IEEE Access*, vol. 11, pp. 145485–145502, 2023.



Xiaodong Yang (Member, IEEE) received the M.S. degree in cryptography from Tongji University, Shanghai, China, in 2005, and the Ph.D. degree in cryptography from Northwest Normal University, Lanzhou, China, in 2010. He worked as a Postdoctoral Researcher with the State Key Laboratory of Cryptology of China in 2016.



Ruiling Li received the B.E. degree in cryptography from Northwest Normal University, Lanzhou, China, in 2023, where she is currently pursuing the M.E. degree in cryptography with the College of Computer Science and Engineering.



Songyu Li received the B.E. degree in cryptography from Northwest Normal University, Lanzhou, China, in 2021, where she is currently pursuing the M.E. degree in cryptography with the College of Computer Science and Engineering.

Her main research interests include cryptography and blockchain.

Na Wang, photograph and biography not available at the time of publication.



Caifen Wang received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2003.

She worked as a Professor with the College of Computer Science and Engineering, Northwest Normal University, Lanzhou, China. She is currently a Professor with the College of Big Data and Internet, Shenzhen Technology University, Shenzhen, China. Her main research interests include public-key cryptography and its applications.